

Liga ProtoFPGA · SAEC 2026

Do código ao chip

Uma introdução prática a FPGAs

Quinta, 08/10 · Lab 124 · Membros da Liga ProtoFPGA

O que vamos fazer

1 O que é uma FPGA

2 Do código à placa

3 Mão na massa: contador

4 Um projeto de verdade: cronômetro

5 Um processador dentro da FPGA

6 Como continuar na liga

Dois jeitos de resolver um problema

Processador

Circuito fixo, feito de fábrica.

Executa as instruções de um programa, uma após a outra.

Exemplos: o chip do celular e do notebook.

FPGA

Chip com milhares de peças lógicas soltas.

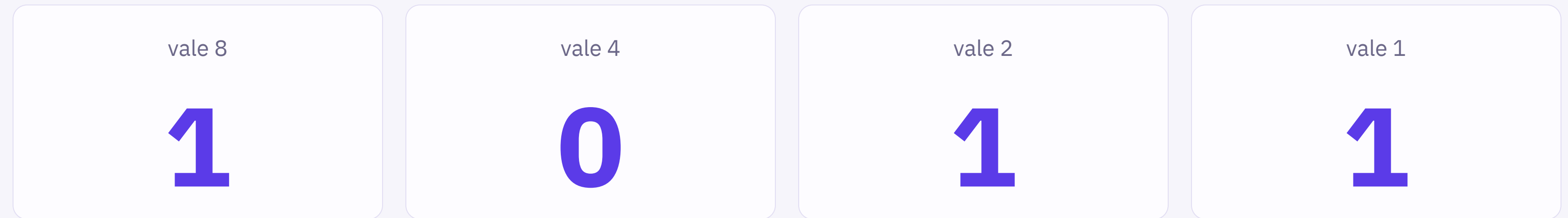
Você descreve o circuito e ele é montado dentro do chip, quantas vezes quiser.

Exemplos: satélites e protótipos de chips.

Os dois são feitos das mesmas peças básicas: portas lógicas e memória.

Antes de tudo

Dentro do chip, tudo é 0 ou 1



$$1011 = 8 + 0 + 2 + 1 = 11$$

Quanto vale cada número?

Converta para decimal

0101

1100

1111

Escreva em binário

6

9

Conferindo

Pergunta	Resposta	Conta
0101	5	$4 + 1$
1100	12	$8 + 4$
1111	15	$8 + 4 + 2 + 1$
6	0110	$4 + 2$
9	1001	$8 + 1$

Portas lógicas

A	B	AND (E)	OR (OU)	XOR (OU exclusivo)
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

NOT (NÃO) inverte: 0 vira 1 e 1 vira 0.

Projete o alarme de um carro

O alarme toca se a **porta estiver aberta** E o **alarme estiver ligado**, OU se o **botão de pânico** for apertado.

Entradas

P = porta aberta · L = alarme ligado · B = botão de pânico

Sua tarefa

Escreva a expressão e desenhe com portas lógicas.

Resposta

Alarme = (P AND L) OR B

P	L	B	Alarme
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Em portas

P e L entram numa porta AND. A saída dela e o B entram numa porta OR.

Em VHDL

```
alarme <= (p and l) or b;
```

Uma linha de código já é um circuito. Das 8 combinações possíveis, o alarme toca em 5.

Como a FPGA vira qualquer porta: a LUT

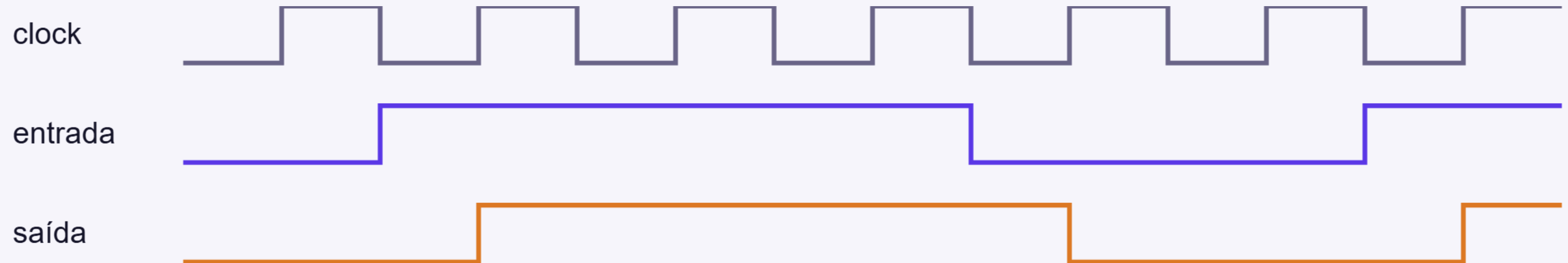
A	B	Saída guardada
0	0	0
0	1	0
1	0	0
1	1	1

A LUT é uma tabelinha guardada na memória. As entradas escolhem a linha, e a saída é o valor guardado.

Trocar a coluna da direita troca a porta. Esta tabela é um AND.

A Artix-7 usa LUTs de 6 entradas: 64 linhas cada.

Flip-flop: a memória de 1 bit



Guarda um valor

Mantém 0 ou 1 até o próximo pulso.

Obedece ao clock

Copia a entrada só na subida do clock.

Base da memória

Contadores e registradores são flip-flops.

O que tem dentro do chip

Blocos lógicos

Milhares de LUTs e flip-flops.

Roteamento

Fios programáveis que ligam os blocos.

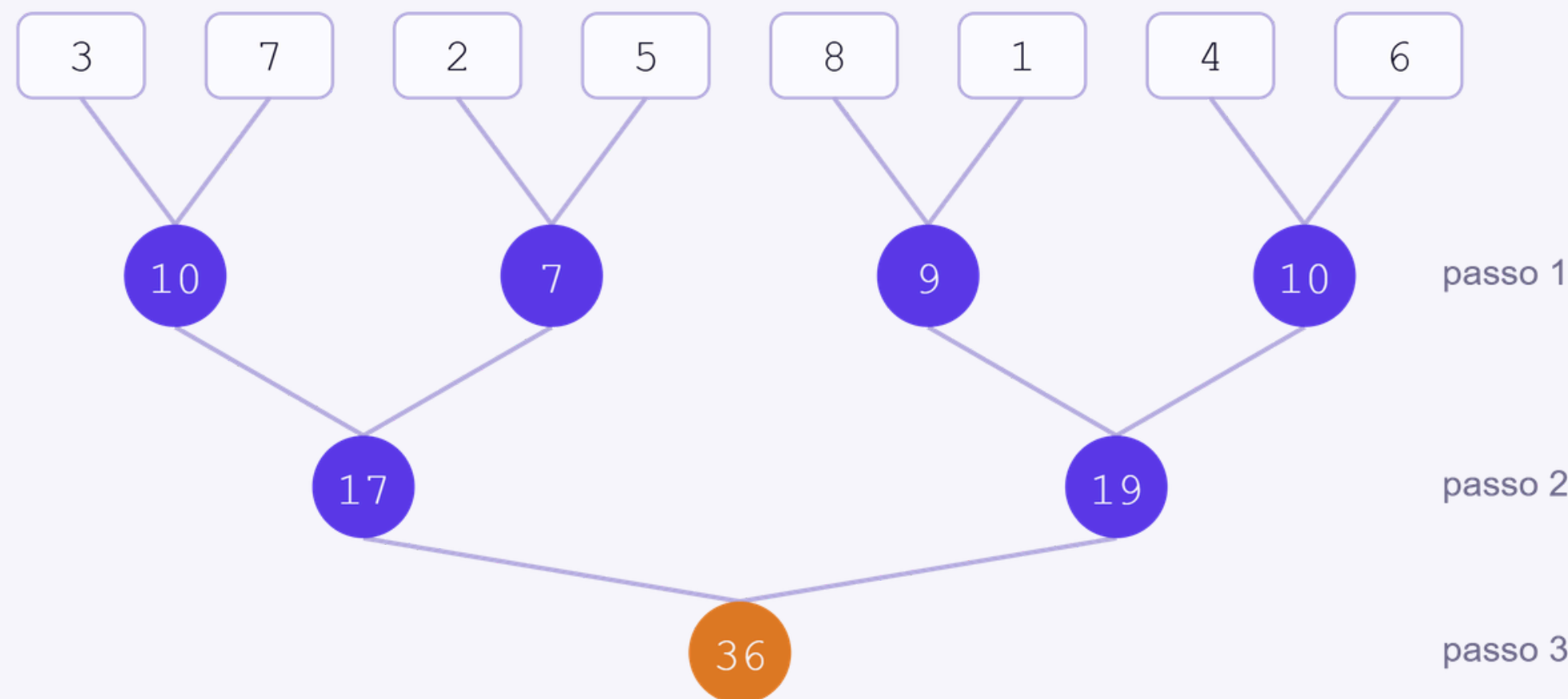
Pinos de entrada e saída

Onde chaves, LEDs e displays se conectam.

Blocos especiais

Memórias (BRAM), multiplicadores (DSP) e geradores de clock.

Somar 8 números: em sequência ou em paralelo



Processador

Uma única unidade de soma (ULA): $3 + 7$, depois $+ 2$, depois $+ 5$... São 7 somas em sequência.

Circuito na FPGA

7 somadores ligados em árvore. As somas de cada nível acontecem juntas: 3 passos.

Com 1024 números

Em sequência: 1023 passos. Em árvore: 10. O custo é ocupar mais área do chip.

Onde se usa

FPGAs estão onde a velocidade importa

Satélites

Processamento a bordo, reprogramável em órbita.

Telecomunicações

Rádio, redes e processamento de sinais.

Veículos

Câmeras, sensores e sistemas de assistência.

Projeto de chips

Testar um chip antes de fabricá-lo.

Quem somos

Liga ProtoFPGA

O que fazemos

Processadores em FPGA, aceleradores de hardware, aritmética de ponto flutuante e prototipação.

Como funciona

Cursos, projetos em grupo e tutoriais abertos no GitHub da liga.

Orientação

Prof. Ney e Prof. Rodrigo, Engenharia de Computação da UFSC Araranguá.

Nossa ferramenta

Conheça a placa Nexys A7

FPGA

Artix-7, da AMD/Xilinx

Entradas

16 chaves e 5 botões

Saídas

16 LEDs e 2 LEDs RGB

Display

8 dígitos de 7 segmentos

Clock

100 MHz

Extras

Sensores, VGA, Ethernet e memória

O caminho é sempre o mesmo

1

Código

VHDL ou Verilog descrevendo o circuito.



2

Síntese

Transforma o código em portas lógicas e flip-flops.



3

Implementação

Escolhe onde cada parte fica no chip e liga os fios.

4

Bitstream

Gera o arquivo .bit com a configuração do chip.



5

Placa

O Hardware Manager grava o .bit e o circuito passa a existir.

Tudo no Vivado

Num projeto pequeno, as etapas 2 a 4 levam alguns minutos.

HDL descreve hardware, não dá ordens

programa em C: a ordem importa

```
x = 2;  
y = x + 1;    // y vale 3  
x = 5;
```

As linhas executam uma depois da outra.

VHDL: a ordem não importa

```
y <= a and b;  
z <= y or c;  
-- trocar as linhas: mesmo circuito
```

Todas as linhas existem ao mesmo tempo, como fios soldados.

As duas linguagens de hardware mais usadas são VHDL e Verilog.

Todo módulo tem duas partes

entity: a caixinha e seus pinos

```
entity alarme is
  port ( p, l, b : in  std_logic;
         saida   : out std_logic );
end alarme;
```

architecture: o que tem dentro

```
architecture rtl of alarme is
begin
  saida <= (p and l) or b;
end rtl;
```

Projeto 1

Cada chave acende um LED

leds.vhd · o circuito

```
entity leds is
  port ( sw  : in  std_logic_vector(15 downto 0);
        led : out std_logic_vector(15 downto 0) );
end leds;

architecture rtl of leds is
begin
  led <= sw;
end rtl;
```

leds.xdc · os pinos

```
set_property -dict { PACKAGE_PIN J15 IOSTANDARD LVCMOS33 }
[get_ports { sw[0] }];

set_property -dict { PACKAGE_PIN H17 IOSTANDARD LVCMOS33 }
[get_ports { led[0] }];
```

PACKAGE_PIN: pino físico do chip · **IOSTANDARD**: nível de tensão (LVCMOS33 = 3,3 V) · **get_ports**: nome do sinal na entity

O que cada linha faz na placa?

adivinha o comportamento

```
led(0) <= not sw(0);  
led(1) <= sw(1) and sw(2);  
led(2) <= sw(3) or sw(4);  
led(3) <= '1';
```

Para cada LED

Diga em que situação ele acende.

Lembre:

not inverte

and: as duas em 1

or: basta uma em 1

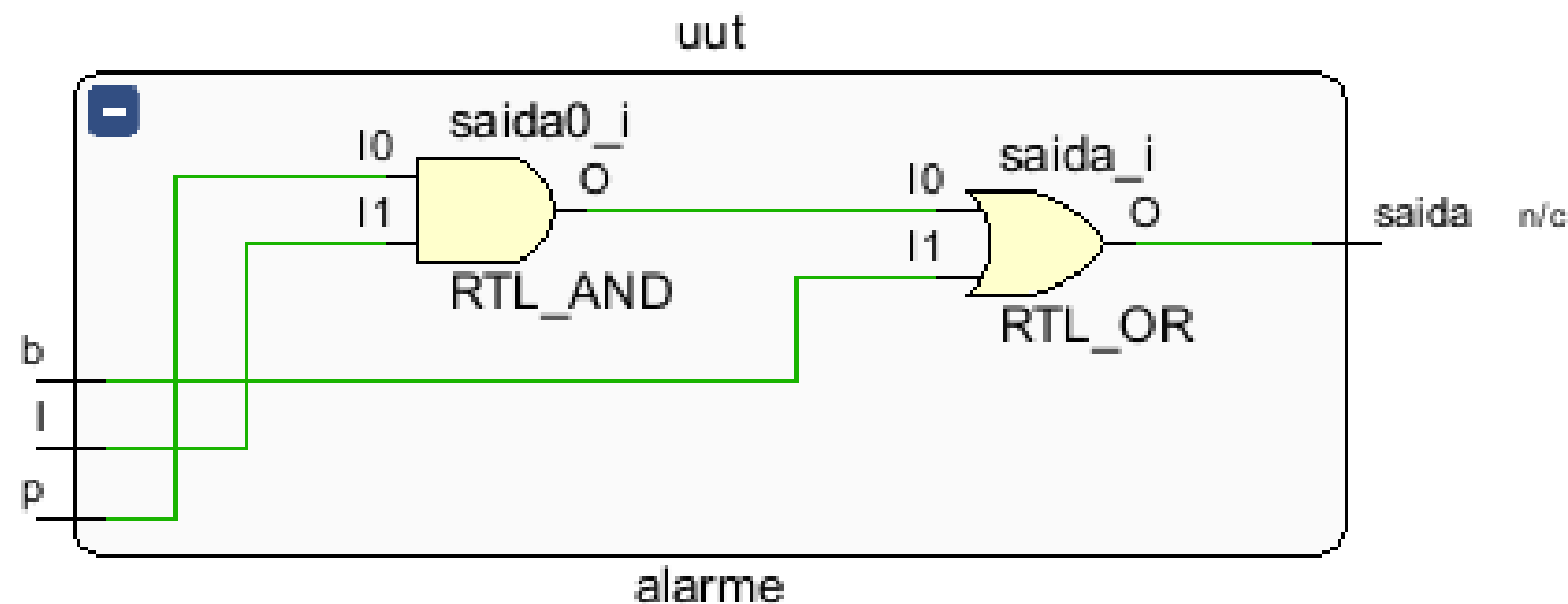
Conferindo

Linha	O que acontece
led(0) <= not sw(0);	LED 0 acende com a chave 0 desligada
led(1) <= sw(1) and sw(2);	LED 1 acende só com as chaves 1 e 2 ligadas
led(2) <= sw(3) or sw(4);	LED 2 acende com a chave 3 ou a 4 ligada
led(3) <= '1';	LED 3 fica sempre aceso

Um passeio pelo Vivado

1. Create Project: escolher a Nexys A7
2. Add Sources: o arquivo .vhd
3. Add Constraints: o arquivo .xdc
4. Run Synthesis
5. Run Implementation
6. Generate Bitstream
7. Hardware Manager: Program Device

O código virou circuito



O que aparece

p e l entram na porta AND; a saída dela e o b entram na porta OR, que gera a saída.

Onde abrir

RTL Analysis → Open Elaborated Design → Schematic

Testar antes de gravar

Simulação

Na placa

Você vê só o resultado final, e cada teste exige minutos de síntese.

Na simulação

Você vê cada sinal, em cada instante, em segundos.

Escrever

o código



Simular

e ver as ondas



Corrigir

se algo der errado



Gravar

na placa

O testbench gera as entradas

tb_alarme.vhd · estímulos

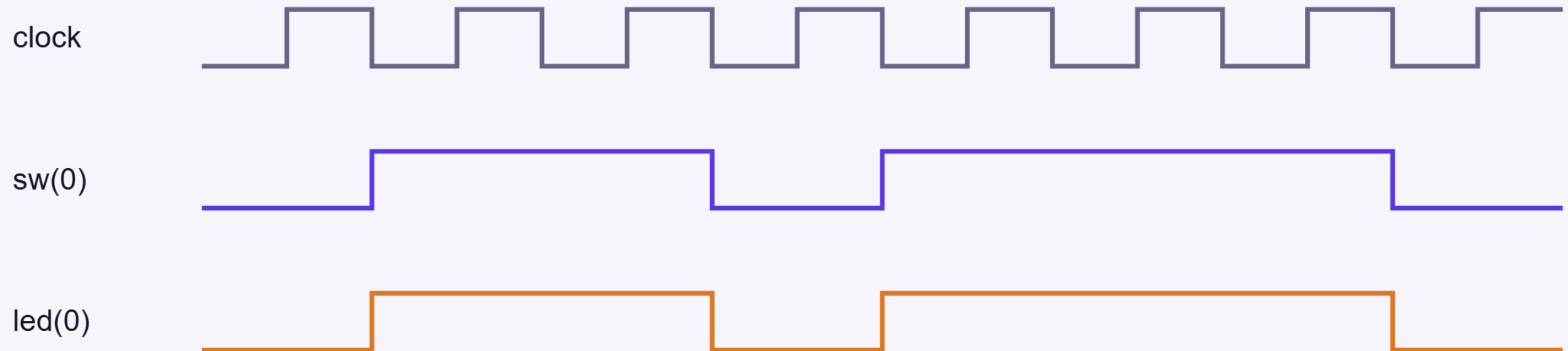
```
process
begin
  p <= '0'; l <= '0'; b <= '0'; wait for 10 ns;
  p <= '1'; l <= '1';          wait for 10 ns;
  p <= '0'; b <= '1';          wait for 10 ns;
  wait;
end process;
```

O testbench é um circuito que só existe na simulação.

Ele muda as entradas ao longo do tempo e deixa a gente observar as saídas.

Testar antes de gravar

Lendo uma forma de onda



Alto = 1, baixo = 0. O tempo corre para a direita.

Preencha a saída do alarme

Lembre: o alarme toca se a **porta estiver aberta (P)** E o **alarme estiver ligado (L)**, OU se o **botão de pânico (B)** for apertado. **Alarme = (P AND L) OR B**

Instante	1	2	3	4	5	6
P (porta)	0	1	1	1	0	0
L (ligado)	0	0	1	1	1	0
B (pânico)	0	0	0	1	0	1
Alarme	?	?	?	?	?	?

A simulação confirma

Instante	1	2	3	4	5	6
P (porta)	0	1	1	1	0	0
L (ligado)	0	0	1	1	1	0
B (pânico)	0	0	0	1	0	1
Alarme	0	0	1	1	0	1

Um contador no display

Divisor de clock

100 milhões de pulsos viram 1 por segundo.



Contador

Soma 1 a cada pulso lento, de 0 a 9999.



Display

Converte para decimal e acende os dígitos.

```
contador_top.vhd · o contador
if tick = '1' then
  if contagem = 9999 then
    contagem <= 0;
  else
    contagem <= contagem + 1;
  end if;
end if;
```

A cada pulso do divisor (tick), a contagem aumenta 1 e volta a 0 depois de 9999. Os três blocos rodam ao mesmo tempo, o tempo todo.

100 milhões de pulsos por segundo

100 MHz

clock da Nexys A7

10 ns

entre um pulso e outro

3 m

o que a luz percorre em 10 ns

Rápido demais para o olho

Se um LED mudasse a cada pulso, a gente nunca veria.

A solução: contar pulsos

A cada 100 milhões de pulsos, passou 1 segundo. Esse é o divisor de clock.

Até quanto o circuito precisa contar?

a)

1 segundo

b)

meio segundo

c)

1 milésimo de segundo

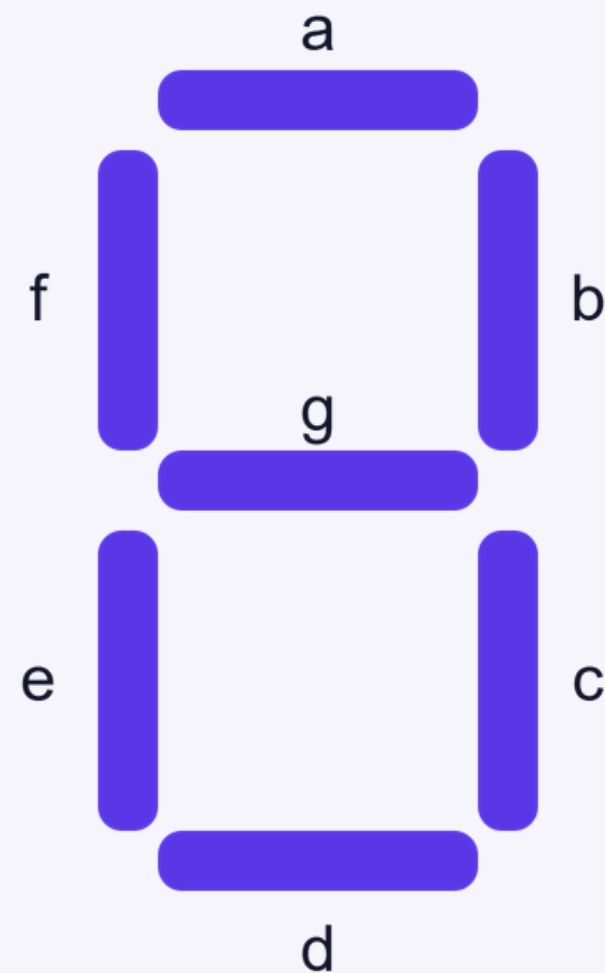
O clock é de 100 MHz: 100 milhões de pulsos por segundo.

Resposta

É só regra de três

Tempo	Pulsos a contar
1 segundo	100.000.000
meio segundo	50.000.000
1 milésimo de segundo	100.000

Como o display mostra um número



São 7 LEDs em forma de 8, chamados de a até g.

Acender certos segmentos forma cada dígito.

Na Nexys A7, o segmento acende com 0, não com 1.

Quais segmentos acendem?

Dígito	a	b	c	d	e	f	g
0							
1							
2							
3							
4							

Marque 1 para aceso. Use o desenho do slide anterior.

A tabela do decodificador

Dígito	a	b	c	d	e	f	g
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2	1	1	0	1	1	0	1
3	1	1	1	1	0	0	1
4	0	1	1	0	0	1	1

Essa tabela é exatamente o que o decodificador faz. Na placa, os valores são invertidos.

8 dígitos, um de cada vez



Neste instante, só o terceiro dígito está aceso. No próximo, só o quarto, e assim por diante.

Fios compartilhados

Os 8 dígitos usam os mesmos 7 fios de segmento.

Um por vez

Cada dígito fica aceso por cerca de 1 milésimo de segundo.

O olho completa

A troca é tão rápida que vemos todos acesos juntos.

O que muda no código

1

Mudar a velocidade

Fácil

2

Contar de trás para frente

Fácil

3

Pausar com uma chave

Fácil

4

Medir o tempo de reação

Demonstração

Como funciona

Os desafios 1 a 3 são explicados no slide: a turma sugere a mudança e você mostra a linha que muda.

Fáceis × demonstração

Nos fáceis muda uma ou duas linhas. O tempo de reação é um projeto pronto, mostrado na placa.

Mudar a velocidade da contagem

O que fazer

Fazer o display contar 4 vezes por segundo em vez de 1.

Na placa Nexys A7

O clock de 100 MHz entra no pino E3 (CLK100MHZ). Nada muda no XDC.

contador_top.vhd • divisor de clock

```
-- antes: um tick por segundo  
constant LIMITE : integer := 100_000_000;  
  
-- depois: quatro ticks por segundo  
constant LIMITE : integer := 25_000_000;
```

Contar de trás para frente

O que fazer

Fazer o contador diminuir: 9999, 9998, 9997...

Na placa Nexys A7

Mesmo display: segmentos CA a CG e dígitos AN0 a AN3. Nada muda no XDC.

contador_top.vhd · contador

```
-- antes
if contagem = 9999 then
    contagem <= 0;
else
    contagem <= contagem + 1;
end if;

-- depois
if contagem = 0 then
    contagem <= 9999;
else
    contagem <= contagem - 1;
end if;
```

Pausar com uma chave

O que fazer

A contagem só anda quando a chave SW0 está ligada.

Na placa Nexys A7

Chave SW0 no pino J15. Adicione a porta SW na entity e a linha da chave no XDC.

contador_top.vhd

```
-- na entity: nova entrada  
SW : in std_logic_vector(0 downto 0);  
  
-- no contador: nova condição  
if tick = '1' and SW(0) = '1' then
```

Medir o tempo de reação

O que fazer

BTNU começa. Depois de 2 s o LED0 acende; aperte BTNC o mais rápido possível. O display mostra o tempo em ms.

Na placa Nexys A7

LED0 (H17), BTNU (M18), BTNC (N17) e o display (CA a CG, AN0 a AN3).

reacao_top.vhd · máquina de estados

```
case estado is
  when ESPERA =>    -- LED apagado por 2 s
    if ms = 1999 then estado <= ACESO; end if;
  when ACESO =>     -- LED aceso, contando ms
    if reagiu = '1' then estado <= MOSTRA; end if;
  when MOSTRA =>    -- display mostra o tempo
    if nova = '1' then estado <= ESPERA; end if;
end case;
```

Intervalo

Voltamos em 15 minutos. Repare na placa sobre a mesa.

Cronômetro de basquete



Contagem regressiva: quando o tempo do quarto zera, o cronômetro para e volta ao estado parado.

Regras do jogo

4 quartos por jogo
FIBA: quartos de 10 minutos
NBA: quartos de 12 minutos

Por dentro do projeto

PI do cronômetro: máquina de estados e contadores
Debounce: um pulso por aperto
Driver do display: multiplexa os dígitos

Como usar o cronômetro na Nexys A7

Botões

CPU RESET reinicia (segure BTNU ao soltar: modo NBA; solto: FIBA)

BTNC para / continua

BTNR carrega o tempo das chaves

BTNU novo quarto

Chaves

SW1–SW0 segundos: 0, 15, 30 ou 45

SW5–SW2 minutos

SW7–SW6 quarto

Saídas

Display segundos.centésimos

LED3–LED0 minutos, em binário

LED7–LED4 quarto (um LED por quarto)

Por que esses controles? O projeto foi feito para a Nexys 2, que tem só 4 botões, 8 chaves e display de 4 dígitos. Por isso os segundos usam só 2 chaves (de 15 em 15 s), o modo é escolhido segurando um botão no reset e o BTNU tem duas funções.

Desenhe a máquina de estados de um semáforo

Básico

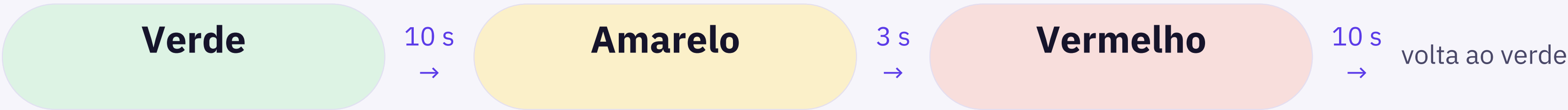
Verde por 10 s, amarelo por 3 s, vermelho por 10 s, e repete.
Desenhe cada estado como um círculo e cada mudança como uma seta.

Desafio

Adicione um botão de pedestre que encurta o verde.

Estado	Luz acesa	Quanto tempo fica	Vai para
1			
2			
3			

O semáforo em três estados



Estado	Luz acesa	Quanto tempo fica	Vai para
Verde	verde	10 s (ou até o botão de pedestre)	Amarelo
Amarelo	amarela	3 s	Vermelho
Vermelho	vermelha	10 s	Verde

Um contador mede o tempo em cada estado e zera a cada mudança, como no cronômetro.

Por que o botão conta várias vezes?



O problema

O contato metálico do botão quica por alguns milissegundos.

A solução

Esperar o sinal ficar estável antes de aceitar o aperto.

Quanto da FPGA o cronômetro ocupou?

0,23%

das LUTs

144 de 63.400

0,12%

dos flip-flops

153 de 126.800

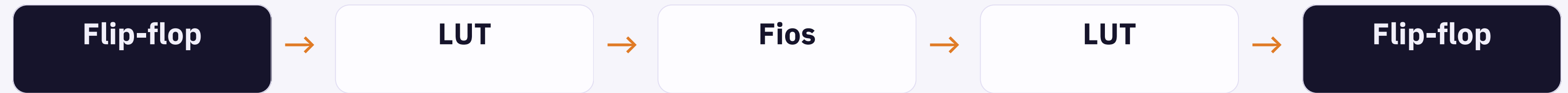
17,6%

dos pinos

37 de 210 (chaves, botões, LEDs e display)

Se sobrou tanto espaço, o que mais caberia aqui?

Quão rápido o circuito pode rodar?



caminho crítico: o sinal sai do flip-flop, passa pelas LUTs e pelos fios do roteamento e precisa chegar antes do próximo pulso de clock

A 100 MHz

O sinal tem 10 ns para ir de um flip-flop ao outro.

Caminho crítico

O caminho mais lento define o clock máximo.

Relatório de timing

O Vivado diz se o projeto aguenta.

Um processador também é um circuito

Descrito em VHDL

O MIPS_S foi feito na universidade, como os projetos de hoje.



Mesmo fluxo

Síntese, implementação e bitstream, como no Projeto 1.



Na FPGA

Vira LUTs, flip-flops e memórias, como qualquer circuito.

32 bits

tamanho dos dados e das instruções

37

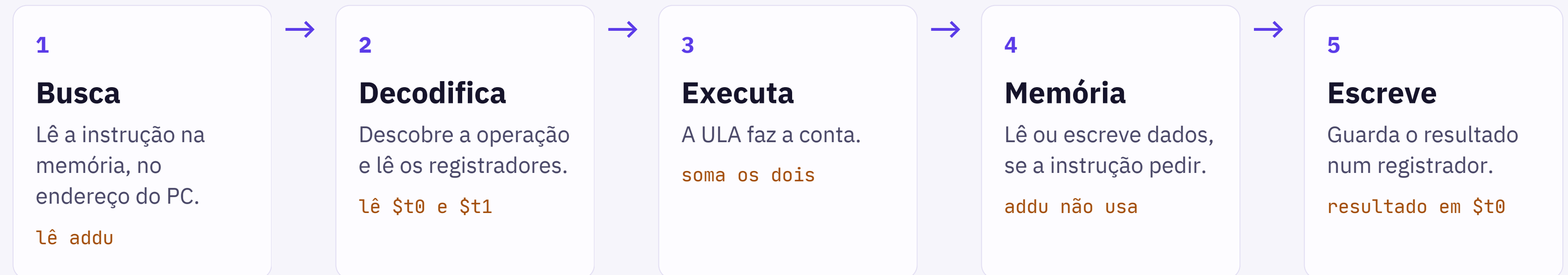
instruções da arquitetura MIPS suportadas

multiciclo

cada instrução leva vários ciclos

Como um processador executa uma instrução

Exemplo: `addu $t0, $t0, $t1` (soma `$t0` com `$t1` e guarda em `$t0`)



No MIPS_S (multiciclo)

Uma etapa por ciclo de clock: cada instrução leva vários ciclos.

Com pipeline

Cada etapa trabalha numa instrução diferente ao mesmo tempo, como uma linha de montagem.

Um programa em assembly

soma.asm · MARS

```
        addiu $t0, $zero, 0    # soma = 0
        addiu $t1, $zero, 1    # i = 1
        addiu $t2, $zero, 11   # limite
loop:    addu  $t0, $t0, $t1     # soma = soma + i
        addiu $t1, $t1, 1      # i = i + 1
        bne   $t1, $t2, loop    # repete se i != 11
```

Registradores

\$t0, \$t1 e \$t2 são pequenas memórias dentro do processador.

bne

"Branch if not equal": volta para loop enquanto i for diferente de 11.

Rode o programa de cabeça

soma.asm

```
        addiu $t0, $zero, 0    # soma = 0
        addiu $t1, $zero, 1    # i = 1
        addiu $t2, $zero, 11   # limite
loop:    addu  $t0, $t0, $t1     # soma = soma + i
        addiu $t1, $t1, 1      # i = i + 1
        bne  $t1, $t2, loop     # repete se i != 11
```

Pergunta 1

Quantas vezes o laço executa?

Pergunta 2

Quanto vale \$t0 no fim?

Resposta

O laço roda 10 vezes e \$t0 termina em 55

10

voltas do laço: i vai de 1 a 10; com i = 11, o bne não volta mais

55

valor final de \$t0 = 1 + 2 + ... + 10

Volta	i	\$t0 (soma)
1	1	1
2	2	3
3	3	6
...	...	...
9	9	45
10	10	55

Um multiplicador melhor deixa o processador mais rápido

Multiplicador serial original

67

ciclos por multiplicação, a 184 MHz

Multiplicador Booth

20

ciclos por multiplicação, a 157 MHz

Ganho real

2,85x

mais rápido na prática

O que a liga está construindo

Representação numérica

IEEE-754, Posit e Takum: precisão e custo em hardware.

IPs de memória

Memórias internas e externas (SPI, DDR) no Vivado e no Quartus.

Scripting com Tcl

Automatizar o fluxo de projeto no Vivado e no Quartus.

Pipeline

MIPS com pipeline em Verilog.

Multiplicador Booth

Acelerar a multiplicação do MIPS_S.

E agora?

Como continuar com a gente

GitHub

github.com/LigaProtoFPGA:
tutoriais e projetos.

Cursos

Programação de baixo nível e
Verilog.

Instagram

@protofpga ·
[instagram.com/protofpga](https://www.instagram.com/protofpga)

Perguntas?

Obrigado! Liga ProtoFPGA · UFSC Araranguá

github.com/LigaProtoFPGA